

Universidade de Caxias do Sul

Curso: Engenharia de Software

Disciplina: Integração de Aplicações

RELATÓRIO TÉCNICO

Sistema de Mensageria com Apache Kafka:

Simulação do Fluxo de Skip de Introdução da Netflix

Professora: Marinês Refosco Severo Bertola

Aluno: Mauricio Knabben

Data de entrega: 30/06/2026

1. Introdução

Este relatório técnico documenta o projeto final da disciplina Integração de Aplicações, desenvolvido com o tema "Sistema de mensageria com Apache Kafka". O projeto simula, em escala reduzida, um dos padrões arquiteturais centrais empregados por plataformas de streaming como a Netflix: a comunicação assíncrona orientada a eventos entre sistemas independentes.

O ponto de partida foi um problema concreto de experiência do usuário: toda vez que o espectador avança para o próximo episódio, o player inicia a reprodução do início, incluindo a introdução da série. O usuário precisa pressionar manualmente o botão "Pular Introdução" novamente. Esse atrito existe porque, no modelo ingênuo, o sistema não sabe de antemão onde o próximo episódio deve começar para aquele usuário específico naquela sessão específica.

A solução implementada elimina esse segundo clique através de pré-computação assíncrona via Kafka: quando o usuário clica em "Próximo Episódio", um evento de domínio é publicado no broker. Um Consumer independente processa esse evento, consulta os metadados do próximo episódio (tempo de término da introdução) e persiste o estado de reprodução no banco de dados. Quando o player carrega o próximo episódio, uma consulta à API retorna exatamente o segundo em que a reprodução deve iniciar — pulando a introdução automaticamente, sem intervenção do usuário.

Além de resolver o problema imediato, o projeto demonstra como o padrão publish-subscribe do Kafka permite que o mesmo evento de domínio seja consumido por múltiplos sistemas independentes, sem que o produtor precise conhecê-los — princípio que sustenta a arquitetura de centenas de microsserviços em plataformas de larga escala.

2. Fundamentação Teórica

2.1 Integração de Aplicações e Integração de Dados

A integração de aplicações trata da comunicação e coordenação entre sistemas distintos, permitindo que troquem informações e acionem comportamentos uns nos outros. A integração de dados, por sua vez, foca na consolidação, transformação e consistência das informações persistidas em diferentes formatos e repositórios. Na prática, ambos os aspectos coexistem: quando sistemas se comunicam, os dados trocados precisam ser transformados entre os formatos de cada sistema.

Neste projeto, a integração de aplicações se manifesta na comunicação assíncrona entre o player web (Vue.js), a API REST (ASP.NET Core) e os consumers Kafka (serviços de background .NET). A integração de dados se manifesta nas sucessivas transformações pelas quais um evento de domínio passa: de record C# para JSON, de JSON para payload binário do protocolo Kafka, de volta para record C# no consumer, de domínio para entidade de persistência, e finalmente para SQL via Entity Framework Core.

2.2 Arquitetura Orientada a Eventos (EDA)

A Arquitetura Orientada a Eventos (Event-Driven Architecture — EDA) é um paradigma no qual os componentes de um sistema se comunicam por meio da produção, detecção e consumo de eventos. Um evento representa um fato imutável que ocorreu no domínio: "o usuário completou o episódio X", "a reprodução iniciou no segundo Y". O produtor do evento não precisa saber quem irá consumi-lo nem o que farão com ele.

Esse desacoplamento é a principal vantagem da EDA em relação à integração síncrona ponto-a-ponto. Em um modelo REST síncrono, o serviço que sinaliza o fim de episódio precisaria chamar explicitamente cada sistema interessado: recomendações, analytics, skip intro, "continue assistindo". Cada novo sistema exigiria alteração no produtor. Com EDA e um broker de mensagens, cada sistema interessado assina o evento de forma independente — o produtor permanece inalterado.

2.3 Apache Kafka: Conceitos Fundamentais

Apache Kafka é uma plataforma distribuída de streaming de eventos. Seus conceitos centrais são:

- **Broker:** servidor que armazena e distribui mensagens, organizadas em tópicos.
- **Tópico (Topic):** canal nomeado para uma categoria de eventos. As mensagens são persistidas em disco e organizadas em partições, permitindo paralelismo.
- **Producer:** componente que publica eventos em um tópico.
- **Consumer:** componente que lê eventos de um tópico. Mantém controle do offset — posição da última mensagem lida.
- **Consumer Group:** grupo de consumers que divide as partições de um tópico entre si, permitindo processamento paralelo com garantia de que cada mensagem é processada por apenas um membro do grupo.
- **Offset:** índice sequencial da mensagem dentro de uma partição. O commit de offset é o mecanismo pelo qual o consumer confirma que processou uma mensagem.

O Kafka difere de filas de mensagens tradicionais (como RabbitMQ no modo queue) por manter as mensagens persistidas por um período configurável, permitindo que múltiplos consumer groups independentes consumam o mesmo tópico de forma isolada — cada um com seu próprio offset.

2.4 Kafka Wire Protocol

A comunicação entre producers, consumers e o broker Kafka ocorre por meio do Kafka Wire Protocol — um protocolo binário proprietário sobre TCP (porta padrão 9092). Diferentemente do protocolo HTTP utilizado na camada REST, o Kafka Wire Protocol é stateful, com multiplexação de requisições, compressão de payload e controle explícito de offset. A biblioteca Confluent.Kafka para C#, utilizada neste projeto, encapsula esse protocolo por meio do wrapper nativo librdkafka, expondo uma API de alto nível. O desenvolvedor não implementa o protocolo diretamente, mas ele está presente em toda comunicação com o broker.

2.5 Integração Síncrona vs. Assíncrona

Na integração síncrona (HTTP/REST), o chamador aguarda a resposta antes de continuar. Isso é adequado quando o cliente precisa do resultado imediatamente — como o player web que precisa saber onde iniciar a reprodução. Na integração assíncrona (Kafka), o produtor publica o evento e segue em frente sem aguardar

processamento. Isso é adequado para operações que podem ser processadas em background — como pré-computar o ponto de início do próximo episódio enquanto o usuário ainda assiste ao episódio atual.

Este projeto demonstra os dois modelos coexistindo: o frontend usa HTTP/REST síncrono para consultar o estado de reprodução, enquanto a atualização desse estado ocorre assincronamente via Kafka. A tensão entre os dois é resolvida por uma estratégia de polling: o frontend consulta o endpoint REST repetidamente até que o Consumer tenha processado o evento.

2.6 ORM como Camada de Transformação de Dados

O Entity Framework Core (EF Core) é um ORM (Object-Relational Mapper) para .NET que atua como middleware de transformação entre o modelo de objetos da aplicação e o modelo relacional do banco de dados. Em termos de integração de dados, o EF Core resolve a incompatibilidade de impedância entre paradigmas: o grafo de objetos da aplicação (hierárquico, com referências) é mapeado para tabelas relacionais (planas, com chaves estrangeiras), e vice-versa.

A comunicação entre o EF Core e o SQL Server ocorre via protocolo TDS (Tabular Data Stream) — protocolo binário proprietário da Microsoft sobre TCP. Assim como o Kafka Wire Protocol, o TDS é abstraído pelo driver (Microsoft.Data.SqlClient) e opera de forma transparente para o desenvolvedor.

3. Problema Proposto

3.1 Contexto

Plataformas de streaming como a Netflix operam com centenas de microsserviços independentes. Quando um usuário interage com o player — avançando episódios, pausando, retomando — múltiplos sistemas precisam ser notificados: o sistema de recomendações deve atualizar sugestões, o analytics deve registrar o comportamento, o sistema de "continue assistindo" deve salvar a posição, e o player deve saber onde iniciar o próximo episódio.

Em uma arquitetura ingênua, cada um desses sistemas seria acionado por chamadas REST síncronas encadeadas a partir do evento do usuário. Esse modelo cria acoplamento direto entre os sistemas, dificulta a adição de novos consumidores sem alterar o produtor, e introduz latência cumulativa na resposta ao usuário.

3.2 O Problema Específico: O Duplo Clique

O problema concreto abordado neste projeto é a fricção gerada pelo fluxo de "Próximo Episódio" em plataformas de streaming:

- O usuário clica em "Próximo Episódio" ao final de um episódio.
- O player carrega o próximo episódio e inicia a reprodução do segundo zero.
- A introdução da série é exibida novamente.
- O usuário precisa clicar manualmente em "Pular Introdução" mais uma vez.

Esse atrito existe porque o sistema não pré-computou onde a reprodução deveria iniciar. O usuário já demonstrou, ao clicar em "Próximo Episódio", que deseja continuar assistindo sem interrupção. A introdução é um conteúdo que ele, quase certamente, já assistiu e não deseja rever.

3.3 Sistemas Envolvidos

O projeto simula a interação entre os seguintes sistemas:

- Player Web (Vue.js): interface do usuário, responsável por reproduzir o vídeo e emitir eventos de interação.
- API REST (ASP.NET Core): ponto de entrada para eventos do player e consultas de estado de reprodução.
- Broker Kafka (Docker): intermediário de mensagens que recebe e distribui eventos de domínio.

- Consumer de Skip Intro (.NET Background Service): processa o evento de fim de episódio e pré-computa o ponto de início do próximo.
- Consumer de Playback (.NET Background Service): processa eventos de início de reprodução e posição, registrando progresso para "continue assistindo".
- Banco de Dados SQL Server (Docker): persiste estados de reprodução e progresso.
- Catálogo de Episódios: repositório de metadados dos episódios, incluindo os timestamps de início e fim de introdução.

3.4 Dificuldades de Integração

As principais dificuldades de integração identificadas são:

- Heterogeneidade de sistemas: o player opera em JavaScript no browser, a API em C# no servidor, o banco em SQL — cada um com modelos de dados distintos.
- Assincronicidade: o usuário não pode aguardar de forma bloqueante o processamento do evento; a pré-computação precisa ocorrer em background.
- Garantia de entrega: o evento de fim de episódio não pode ser perdido — se o Consumer falhar, deve ser possível reprocessar.
- Idempotência: o mesmo evento não deve gerar múltiplos estados de reprodução, mesmo que seja entregue mais de uma vez.
- Sessão: o estado de reprodução pré-computado deve ser válido apenas para a sessão específica que o gerou, evitando que um skip de uma sessão anterior seja aplicado numa nova abertura do episódio.

3.5 Justificativa da Solução

O Apache Kafka foi escolhido como tecnologia central de integração pelas seguintes razões:

- Desacoplamento: o player publica o evento e não precisa conhecer os consumers. Novos sistemas (recomendações, analytics, conquistas) podem ser adicionados sem alterar o produtor.
- Múltiplos consumers independentes: diferentes consumer groups podem consumir o mesmo tópico com offsets independentes, cada um processando o evento para sua finalidade.
- Garantia de entrega: com commit manual de offset (at-least-once delivery), um evento nunca é perdido — em caso de falha, é reprocessado.
- Persistência: mensagens ficam retidas no tópico por tempo configurável, permitindo reprocessamento e auditoria.
- Escalabilidade: particionamento permite distribuir a carga entre múltiplas instâncias do Consumer.

4. Análise da Solução

4.1 Visão Geral do Fluxo

A solução se estrutura em dois fluxos principais: o fluxo de pré-computação (assíncrono, via Kafka) e o fluxo de consulta (síncrono, via REST).

No fluxo de pré-computação: quando o usuário clica em "Próximo Episódio", o player chama o endpoint `POST /episodes/{id}/completed`. A API cria um `EpisodeCompletedEvent` com os dados da sessão e o publica no tópico `episode.completed` do Kafka. O Consumer do grupo `skip-intro-group` lê o evento, consulta o catálogo para obter o tempo de término da introdução do próximo episódio, e persiste um `PlaybackState` com o campo `StartAtSeconds` já calculado.

No fluxo de consulta: o player realiza polling no endpoint `GET /episodes/{id}/playback-state` até que a API retorne um estado com `source` igual a `"pre-computed"`. Ao receber esse estado, o player salta diretamente para o segundo indicado, eliminando o segundo clique.

4.2 Por que Não REST Síncrono

Uma alternativa seria a API calcular sincronicamente o ponto de início ao receber o `POST /completed` e retornar o resultado imediatamente. Essa abordagem tem limitações estruturais importantes: acopla o tempo de resposta da API ao tempo de processamento; impede que outros sistemas reajam ao mesmo evento sem alteração na API; viola o princípio de responsabilidade única ao misturar publicação de evento com processamento; e não escala bem quando o processamento envolve chamadas a sistemas externos.

O Kafka permite que o `POST /completed` retorne imediatamente (HTTP 202 Accepted), enquanto o processamento ocorre em background, de forma independente.

4.3 Extensibilidade: Múltiplos Consumers

A arquitetura de publish-subscribe do Kafka permite que o mesmo evento `EpisodeCompletedEvent` seja consumido por múltiplos consumer groups independentes, cada um com sua finalidade. No projeto, dois consumers já estão

implementados. A tabela abaixo ilustra como a mesma estrutura suportaria os sistemas de uma plataforma real:

Consumer Group	Tópico Consumido	Ação
skip-intro-group	episode.completed	Pré-computa ponto de início do próximo episódio
analytics-group	episode.completed	Registra conclusão para métricas de engajamento
recommendation-group	episode.completed	Atualiza sugestões do usuário
watch-progress-group	playback.position	Salva posição para "continue assistindo"
playback-control-group	playback.started	Marca PlaybackState como consumido (idempotência)

Nenhum desses grupos requer alteração no produtor (API/player). Isso demonstra na prática o princípio de desacoplamento que justifica o Kafka como escolha de integração.

4.4 Decisões de Design

SessionId como âncora de sessão: cada abertura do player gera um UUID de sessão único. Todos os eventos publicados naquela sessão carregam esse ID. O Consumer só aceita um PlaybackState para uma combinação (UserId, EpisodeId, SessionId), e o estado expira em 2 horas. Isso garante que um skip pré-computado numa sessão anterior não seja aplicado equivocadamente numa nova abertura do episódio.

Fallback no GET playback-state: se nenhum estado pré-computado for encontrado (acesso direto ao episódio, sem ter vindo do anterior), a API retorna StartAtSeconds: 0, source: catalog-default. O player sempre tem uma resposta válida, sem erros.

At-least-once delivery com idempotência: o Consumer usa commit manual de offset (EnableAutoCommit = false), garantindo que eventos não sejam perdidos em caso de falha. A idempotência é garantida por um índice único (UserId, EpisodeId) na tabela PlaybackStates, que converte tentativas de inserção duplicada em updates.

5. Arquitetura da Integração

5.1 Estrutura de Projetos

O projeto é organizado em seis módulos com responsabilidades bem definidas, seguindo os princípios de separação de responsabilidades e inversão de dependência:

Projeto	Responsabilidade
SharedKernel	Eventos de domínio, modelos e interfaces de repositório. Não tem dependências dos demais projetos.
Catalog	Entidade Episode com metadados (intro start/end seconds, videoStorageKey) e interface ICatalogRepository.
Application	Serviços de aplicação (EpisodeService, CatalogService) e DTOs. Orquestra a lógica sem conhecer infraestrutura.
Infrastructure	EF Core, DbContext, repositórios SQL Server, Consumers Kafka como BackgroundServices.
Streaming	EpisodeProducer e PlaybackEventProducer — publicação de eventos nos tópicos Kafka.
API	Controllers REST (EpisodesController, CatalogController). Ponto de entrada HTTP da solução.

5.2 Fluxo de Integração

O fluxo completo de integração, do clique do usuário até o skip automático, é:

Etapa	Componente	Protocolo / Mecanismo
1	Player clica em Próximo Episódio	Evento de UI (JavaScript)
2	POST /episodes/{id}/completed	HTTP/REST — JSON
3	API cria EpisodeCompletedEvent e publica no tópico episode.completed	Kafka Wire Protocol (TCP binário)
4	EpisodeConsumer lê o evento do tópico	Kafka Wire Protocol — commit manual de offset
5	Consumer consulta catálogo (introEndSeconds do próximo episódio)	EF Core — TDS sobre TCP
6	Consumer persiste PlaybackState com StartAtSeconds calculado	EF Core — TDS sobre TCP

7	Player faz polling: GET /episodes/{id}/playback-state	HTTP/REST — JSON
8	API retorna { startAtSeconds: 70, source: 'pre-computed' }	HTTP/REST — JSON
9	Player salta para o segundo indicado e inicia reprodução	HTMLVideoElement.currentTime
10	Player publica PlaybackStartedEvent	HTTP/REST → Kafka Wire Protocol
11	PlaybackConsumer marca PlaybackState como consumido (IsConsumed = true)	EF Core — TDS sobre TCP

5.3 Tópicos Kafka e Eventos de Domínio

Três eventos de domínio trafegam pelo sistema, cada um em seu tópico:

Evento	Tópico	Campos Relevantes
EpisodeCompletedEvent	episode.completed	UserId, EpisodeId, NextEpisodeId, SessionId, CompletedAt
PlaybackStartedEvent	playback.events	UserId, EpisodeId, StartAtSeconds, SessionId, StartedAt
PlaybackPositionEvent	playback.events	UserId, EpisodeId, PositionSeconds, SessionId, OccurredAt (heartbeat a cada 30s)

5.4 Infraestrutura Docker

Toda a infraestrutura é provisionada via Docker Compose, permitindo reproduzir o ambiente com um único comando:

Serviço	Porta	Função
zookeeper	2181	Coordenação do cluster Kafka via protocolo ZAB (ZooKeeper Atomic Broadcast)
kafka	9092	Broker Kafka — recebe e distribui eventos via Kafka Wire Protocol
kafka-ui	8090	Interface web para inspeção de tópicos, offsets e mensagens
sqlserver	1433	Banco de dados relacional — comunicação via protocolo TDS
API	5000	API REST ASP.NET Core + player web Vue.js (arquivos estáticos)

5.5 Camadas de Protocolo

O projeto utiliza simultaneamente três camadas de protocolo de comunicação, evidenciando a heterogeneidade típica de sistemas integrados:

Camada	Protocolo	Formato de Dados	Uso
Frontend ↔ API	HTTP/1.1 (REST)	JSON (UTF-8)	Consultas e comandos do player
API ↔ Kafka Broker	Kafka Wire Protocol (TCP binário)	JSON serializado no payload	Publicação e consumo de eventos
API / Consumer ↔ SQL Server	TDS — Tabular Data Stream (TCP binário)	Binário estruturado	Persistência via EF Core
ZooKeeper ↔ Kafka	ZAB — ZooKeeper Atomic Broadcast	Binário interno	Coordenação do cluster

6. Tecnologias Utilizadas

6.1 Apache Kafka

Kafka é a peça central da solução. Escolhido pelo modelo publish-subscribe, persistência de mensagens, suporte a múltiplos consumer groups independentes e garantias de entrega configuráveis. A versão utilizada é compatível com o cliente Confluent.Kafka, que internamente usa a biblioteca nativa librdkafka para implementar o Kafka Wire Protocol.

6.2 Confluent.Kafka para C#

Biblioteca oficial da Confluent para integração de aplicações .NET com Apache Kafka. Encapsula o Kafka Wire Protocol via librdkafka, expondo uma API de alto nível para produção e consumo de mensagens. Configura-se com bootstrap servers, consumer group, política de offset e commit manual — todos conceitos do protocolo Kafka, transparentes para o código de aplicação.

6.3 .NET 10 e ASP.NET Core

Plataforma de desenvolvimento para a API REST e os consumers Kafka. Os consumers são implementados como IHostedService (BackgroundService), ciclo de vida gerenciado pelo host genérico do .NET. Escolhido pela maturidade do ecossistema para aplicações de backend e pela integração nativa com Entity Framework Core.

6.4 Entity Framework Core (EF Core)

ORM responsável pela integração entre o modelo de objetos .NET e o banco de dados relacional SQL Server. Realiza o mapeamento das entidades PlaybackStateEntity e WatchProgressEntity para as tabelas correspondentes, traduzindo operações CRUD em comandos SQL via protocolo TDS. Representa a camada de transformação de dados entre o paradigma orientado a objetos e o paradigma relacional.

6.5 SQL Server

Banco de dados relacional para persistência dos estados de reprodução e progresso de visualização. A comunicação ocorre via protocolo TDS (Tabular Data

Stream) na porta 1433. Provisionado via Docker com scripts de criação de banco, tabelas e seed de dados.

6.6 Vue.js 3

Framework JavaScript progressivo para o player web. Responsável pela interface do usuário: reprodução do vídeo via HTMLVideoElement, exibição do botão "Pular Introdução" durante a janela da intro, botão "Próximo Episódio", barra de progresso do processamento Kafka e console de mensageria em tempo real. Comunica-se com a API via fetch API (HTTP/REST/JSON).

6.7 ZooKeeper

Serviço de coordenação distribuída que gerencia o cluster Kafka via protocolo ZAB (ZooKeeper Atomic Broadcast). Responsável pela eleição de líder entre brokers e gerenciamento de metadados do cluster. Não requer configuração manual neste projeto — provisionado automaticamente via Docker Compose.

6.8 Docker e Docker Compose

Toda a infraestrutura (Kafka, ZooKeeper, Kafka UI, SQL Server) é provisionada via Docker Compose. Isso garante reprodutibilidade do ambiente, isolamento de rede entre os serviços e facilidade de inicialização com um único comando.

6.9 System.Text.Json

Biblioteca nativa do .NET para serialização e desserialização JSON. Usada para converter os eventos de domínio (records C#) em strings JSON que compõem o payload das mensagens Kafka, e para desserializar as mensagens recebidas pelos consumers de volta para os tipos de domínio.

6.10 Justificativa Técnica das Escolhas

A escolha de Kafka sobre alternativas como RabbitMQ se justifica pelo modelo de persistência de mensagens (RabbitMQ remove a mensagem após consumo, Kafka a mantém), pela capacidade de múltiplos consumer groups independentes no mesmo tópico, e pela escalabilidade horizontal via particionamento — características alinhadas ao cenário de múltiplos sistemas consumindo o mesmo evento de domínio.

A escolha de .NET com EF Core e SQL Server reflete a integração coesa entre as tecnologias: o ecossistema oferece suporte de primeira classe a Kafka (via Confluent.Kafka), ORM maduro e pool de conexões otimizado para o protocolo TDS.

7. Implementação

7.1 Publicação de Eventos (Producer)

Quando o usuário clica em "Próximo Episódio", o frontend chama POST /episodes/{id}/completed. O controller cria um EpisodeCompletedEvent e delega a publicação ao EpisodeProducer:

```
var evento = new EpisodeCompletedEvent(  
    UserId:      userId == Guid.Empty ? Guid.NewGuid() : userId,  
    EpisodeId:   episodeId,  
    NextEpisodeId: episodeId + 1,  
    SessionId:   sessionId == Guid.Empty ? Guid.NewGuid() :  
sessionId,  
    CompletedAt: DateTime.UtcNow  
);  
await _episodeProducer.PublishEpisodeCompletedAsync(evento);  
return Accepted(new { userId = evento.UserId, sessionId =  
evento.SessionId, ... });
```

O retorno imediato com HTTP 202 Accepted é fundamental: o usuário não aguarda o processamento do Consumer.

7.2 Consumo e Pré-computação (Consumer)

O EpisodeConsumer roda como BackgroundService singleton. Para resolver DbContext (Scoped) dentro de um Singleton, utiliza IServiceScopeFactory — padrão obrigatório com EF Core em BackgroundServices:

```
private void ProcessEvent(EpisodeCompletedEvent evento)  
{  
    using var scope = _scopeFactory.CreateScope();  
    var catalogRepo =  
scope.ServiceProvider.GetRequiredService<ICatalogRepository>();  
    var playbackRepo =  
scope.ServiceProvider.GetRequiredService<IPlaybackStateRepository>(  
);  
  
    var nextEpisode = catalogRepo.GetById(evento.NextEpisodeId);  
    var startAt      = nextEpisode.IntroEndSeconds; // onde a intro  
termina  
  
    var state = new PlaybackState(  

```



```

        UserId:         evento.UserId,
        EpisodeId:      evento.NextEpisodeId,
        StartAtSeconds: startAt,
        SessionId:      evento.SessionId,
        ExpiresAt:      DateTime.UtcNow.AddHours(2)
    );
    playbackRepo.SaveAsync(state).GetAwaiter().GetResult();
}

```

O commit manual de offset ocorre após o processamento bem-sucedido, garantindo at-least-once delivery:

```

var msg = _consumer.Consume(ct);
var evento =
    JsonSerializer.Deserialize<EpisodeCompletedEvent>(msg.Message.Value);
if (evento is not null) ProcessEvent(evento);
_consumer.Commit(msg); // só comita após processar com sucesso

```

7.3 Consulta de Estado e Fallback

O endpoint GET /episodes/{id}/playback-state implementa a lógica de fallback: retorna o estado pré-computado se disponível e válido para a sessão, ou o default do catálogo (startAtSeconds: 0) caso contrário:

```

// Resposta com skip: { startAtSeconds: 70, source: 'pre-computed' }
// Resposta sem skip: { startAtSeconds: 0, source: 'catalog-default' }

```

O frontend realiza polling neste endpoint em intervalos de 1 segundo, com timeout de 12 tentativas. Ao receber source: 'pre-computed', cancela o polling e salta para o segundo indicado.

7.4 Heartbeat de Posição e Playback Iniciado

Dois eventos adicionais demonstram a extensibilidade da arquitetura:

- **PlaybackPositionEvent**: publicado a cada 30 segundos enquanto o usuário assiste, registrando a posição atual no tópico playback.events. O Consumer correspondente salva a posição na tabela WatchProgress — base para a funcionalidade "continue assistindo".

- **PlaybackStartedEvent**: publicado quando a reprodução inicia. O Consumer marca o **PlaybackState** como **IsConsumed = true**, garantindo que o mesmo skip não seja aplicado numa segunda abertura do episódio na mesma sessão.

7.5 Pipeline de Transformação de Dados

O fluxo de dados do **EpisodeCompletedEvent** ilustra a cadeia de transformações que conecta os sistemas:

Etapa	Representação	Tecnologia
1 — Criação	Record C# (EpisodeCompletedEvent)	C# / .NET
2 — Serialização	String JSON: {"UserId":"...", "EpisodeId":1,...}	System.Text.Json
3 — Transporte	Payload binário no Kafka Wire Protocol	Confluent.Kafka / librdkafka
4 — Desserialização	Record C# (EpisodeCompletedEvent)	System.Text.Json
5 — Mapeamento de domínio	Objeto PlaybackState	Lógica de aplicação
6 — Mapeamento ORM	PlaybackStateEntity (EF Core)	Entity Framework Core
7 — Persistência	Linha SQL na tabela PlaybackStates	SQL Server via TDS

Cada etapa representa uma transformação de formato — do modelo de domínio para JSON, de JSON para binário, de binário de volta para domínio, de domínio para entidade relacional, de entidade para SQL. Esse pipeline exemplifica na prática o conceito de integração de dados por transformação.

7.6 Console de Mensageria

O frontend inclui um console de mensageria em tempo real que exibe todos os eventos do fluxo: publicação do **EpisodeCompletedEvent**, tentativas de polling, resposta do Consumer, seek no vídeo e início da reprodução. Esse componente foi desenvolvido para fins didáticos, tornando visível o fluxo assíncrono que normalmente ocorre de forma transparente.

8. Testes e Resultados

8.1 Fluxo de Teste Principal

O projeto inclui o arquivo `NetflixSkipIntroMensageria.API.http`, compatível com o cliente HTTP do Visual Studio e com ferramentas como o Postman. Esse arquivo concentra todas as requisições necessárias para testar o backend de forma isolada, sem depender do player web. O fluxo completo é testado seguindo a sequência abaixo, utilizando o arquivo `.http` ou o player web diretamente:

Passo	Ação	Resultado Esperado
1	GET /catalog/episodes	Lista de 5 episódios com metadados (introStartSeconds, introEndSeconds)
2	POST /episodes/1/completed	HTTP 202 com userId e sessionId gerados; evento publicado no Kafka
3	(aguardar ~1s para Consumer processar)	Consumer lê evento, consulta EP2, salva PlaybackState com startAtSeconds: 70
4	GET /episodes/2/playback-state?userId=...&sessionId=...	{ startAtSeconds: 70, source: 'pre-computed' }
5	GET /episodes/3/playback-state (acesso direto, sem sessionId válido)	{ startAtSeconds: 0, source: 'catalog-default' }

8.2 Evidências de Funcionamento

As seguintes evidências demonstram o funcionamento do sistema:

- Kafka UI (<http://localhost:8090>): exibe o tópico `episode.completed` com a mensagem publicada, offset 0, e o consumer group `skip-intro-group` com lag 0 após processamento.
- Console de mensageria do player: sequência de logs mostrando `EpisodeCompletedEvent` publicado, tentativas de poll, resposta `pre-computed` recebida, `seek` executado no vídeo.
- SQL Server (tabela `PlaybackStates`): registro inserido com `UserId`, `EpisodeId` = 2, `StartAtSeconds` = 70, `SessionId`, `ExpiresAt` e `IsConsumed` = false antes do início, true após.
- Reprodução automática: o player inicia o EP2 no segundo 70 sem necessidade de interação do usuário.

8.3 Cenários Adicionais Testados

- Acesso direto: abrir o EP3 sem ter vindo do EP2. Retorna source: catalog-default, startAtSeconds: 0. Player inicia do zero — comportamento correto.
- Sessão expirada (simulado): estado com ExpiresAt no passado retorna catalog-default. Prevenção de skip inválido.
- Estado já consumido: após o PlaybackStartedEvent ser processado, IsConsumed = true. Nova consulta retorna catalog-default, impedindo que o skip seja reaplicado.
- Timeout de polling: se o Consumer não processar em 12 tentativas (~12s), o player carrega o próximo episódio do zero, com degradação graciosa.

8.4 Limitações Encontradas

As seguintes limitações foram identificadas no escopo do projeto:

- Vídeo único: o projeto usa um único arquivo de vídeo (GTAVI.mp4) para todos os episódios, simulando um CDN. Em produção, cada episódio teria seu próprio arquivo e URL de storage.
- Sem autenticação: userId é gerado aleatoriamente por sessão de browser. A integração com sistema de autenticação (JWT) está prevista como extensão futura.
- Polling em vez de WebSocket: a estratégia de polling, embora funcional, introduz latência variável de até 1 segundo. WebSockets eliminariam esse delay, mas adicionariam complexidade ao escopo do projeto.
- NextEpisodeId fixo: o ID do próximo episódio é calculado como episodeId + 1, sem consulta ao catálogo para verificar se pertence à mesma série/temporada.
- Sem retry explícito no Consumer: em caso de falha no processamento, o commit não é feito e a mensagem é reprocessada (at-least-once), mas não há fila de dead-letter para mensagens que falham repetidamente.

9. Conclusão

Este projeto demonstrou na prática como um sistema de mensageria baseado em Apache Kafka resolve problemas de integração entre sistemas heterogêneos de forma desacoplada, assíncrona e extensível. Partindo de um problema concreto de experiência do usuário — o duplo clique para pular a introdução — a solução implementou um pipeline de eventos que conecta um player web JavaScript, uma API REST C#, consumers .NET e um banco de dados SQL Server, utilizando simultaneamente os protocolos HTTP/REST, Kafka Wire Protocol e TDS.

O principal aprendizado arquitetural é que o evento de domínio `EpisodeCompletedEvent` carrega um significado que transcende seu consumidor imediato. A mesma notificação de "usuário completou o episódio" que dispara o pré-cálculo do skip de intro é a mesma que alimentaria sistemas de recomendação, analytics e conquistas — sem que o produtor precise conhecer qualquer um deles. Esse é o princípio central que sustenta arquiteturas de larga escala como a da Netflix.

A combinação de integração de aplicações (Kafka, REST, Vue.js) e integração de dados (EF Core como camada de transformação objeto-relacional, serialização JSON dos eventos, pipeline de transformação de formatos) cobriu múltiplos cenários da proposta do projeto, demonstrando como esses dois aspectos coexistem em sistemas reais.

Como extensões naturais do projeto estão: autenticação JWT para associar eventos a usuários reais, WebSockets para eliminar o polling, fila de dead-letter para mensagens com falha repetida, e integração com um sistema de recomendações que consuma o mesmo tópico `episode.completed` — evidenciando o desacoplamento na prática com a adição de um novo consumer sem alterar uma linha do produtor.

10. Referências Bibliográficas

APACHE SOFTWARE FOUNDATION. Apache Kafka Documentation. Disponível em: <https://kafka.apache.org/documentation/>. Acesso em: jun. 2026.

CONFLUENT. Confluent.Kafka .NET Client Documentation. Disponível em: <https://docs.confluent.io/clients-confluent-kafka-dotnet/current/overview.html>. Acesso em: jun. 2026.

MICROSOFT. Entity Framework Core Documentation. Disponível em: <https://learn.microsoft.com/pt-br/ef/core/>. Acesso em: jun. 2026.

MICROSOFT. ASP.NET Core — BackgroundService e IHostedService. Disponível em: <https://learn.microsoft.com/pt-br/aspnet/core/fundamentals/host/hosted-services>. Acesso em: jun. 2026.

FOWLER, Martin. Event-Driven Architecture. martinowler.com. Disponível em: <https://martinfowler.com/articles/201701-event-driven.html>. Acesso em: jun. 2026.

NETFLIX TECHNOLOGY BLOG. Kafka at Netflix: Billions of Messages Per Day. Disponível em: <https://netflixtechblog.com/kafka-inside-keystone-pipeline-dd5aeabaf6bb>. Acesso em: jun. 2026.

HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley, 2003.

APACHE ZOOKEEPER. ZooKeeper: A Distributed Coordination Service for Distributed Applications. Disponível em: <https://zookeeper.apache.org/doc/current/zookeeperOver.html>. Acesso em: jun. 2026.